DIRECTA V2

MÓDULO GERENTE GENERAL

INFORME Y PLAN DE PRUEBAS

Versión 1.0 · 25 de octubre de 2026

Rodrigo Bello · Ignacio González · Benjamín Gutiérrez · Javiera Villalobos

Revisión del código fuente y registro de pruebas funcionales informadas

1. Objetivo

Documentar la evidencia disponible sobre las pruebas de DIRECTA V2, distinguir los controles identificados en el código de las pruebas ejecutadas y establecer un plan de verificación para el módulo Gerente General. Se considera el criterio de evaluación del entregable: existe evidencia de pruebas cuando hay un plan de pruebas o pruebas reales en el código; las pruebas de ejemplo del framework no se contabilizan.

2. Material examinado y método

Se examinó el archivo proporcionado proyecto_directa_MVP.rar. Contiene nueve archivos Python principales (main.py, agentes.py, ia_config.py, dashboard.py, guia.py, indicadores.py, memoria.py, modulos_base.py y test_nvidia.py), además de interfaces HTML y archivos auxiliares. La revisión incluyó inventario, búsqueda de archivos y funciones de prueba, lectura de la configuración de proveedores y análisis sintáctico de los nueve archivos Python.

La revisión se realizó de manera estática, sin iniciar el servidor ni enviar solicitudes a Groq o NVIDIA. No se utilizaron las credenciales contenidas en el archivo de configuración ni se verificó la disponibilidad de servicios externos. Las observaciones de funcionamiento aportadas por el equipo se consignan como pruebas manuales informadas, sin actas o capturas adjuntas.

3. Resultado de la revisión del código

Hallazgo	Evidencia y alcance
Prueba de conexión NVIDIA	test_nvidia.py envía una solicitud HTTP al servicio de chat de NVIDIA y muestra código de estado y respuesta. Es una prueba técnica real definida por el proyecto; no se ejecutó en esta revisión.
Pruebas unitarias automatizadas	No se identificaron suites pytest/unittest ni funciones test_ adicionales entre los archivos Python entregados.
Análisis sintáctico	Los nueve archivos .py se analizaron con ast.parse sin errores de sintaxis. Esta verificación no demuestra ejecución funcional.
Resiliencia de IA	ia_config.py implementa rotación de claves Groq, límites de espera, reintentos y tratamiento de respuestas HTTP. La existencia del código no equivale a pruebas exitosas de recuperación.
Agentes	agentes.py define y valida la estructura de agentes; la comprobación de su ejecución completa requiere correr el flujo real.

4. Pruebas funcionales informadas por el equipo

Los casos siguientes reflejan comportamientos observados y descritos por el equipo. No se dispone de evidencia fechada ni de resultados repetibles suficientes para declararlos formalmente aprobados.

PF-01 · Carga e indexación

Procedimiento: Subir un documento y verificar su indexación vectorial.

Resultado informado o esperado: Entre 2 y 5 minutos según tamaño del documento.

Estado de evidencia: Observado por el equipo; sin registro adjunto.

PF-02 · Análisis de agentes

Procedimiento: Verificar ejecución secuencial de 14 agentes.

Resultado informado o esperado: Avance visible en terminal, con mensajes OK y alertas ante problemas.

Estado de evidencia: Observado por el equipo; sin captura adjunta.

PF-03 · Tiempo total

Procedimiento: Cronometrar indexación y análisis completo.

Resultado informado o esperado: Habitualmente alrededor de 6 minutos; interfaz estima cerca de 13.

Estado de evidencia: Estimación reportada; sin serie de mediciones.

PF-04 · Recuperación vectorial

Procedimiento: Realizar consultas tras procesar el documento inicialmente.

Resultado informado o esperado: La información se reutiliza sin repetir todo el análisis inicial.

Estado de evidencia: Comportamiento informado; sin evidencia técnica de consulta.

PF-05 · Error de proveedor

Procedimiento: Provocar o detectar una consulta sin respuesta de la API.

Resultado informado o esperado: La aplicación puede quedarse esperando; se requiere repetir la consulta.

Estado de evidencia: Incidencia conocida; recuperación no aprobada.

PF-06 · Rotación y respaldo

Procedimiento: Validar rotación de cuatro claves Groq y alternativa NVIDIA.

Resultado informado o esperado: La configuración está presente en ia_config.py.

Estado de evidencia: Identificado en código; prueba de fallo/conmutación pendiente.

5. Plan de pruebas complementarias

PT-01 · Conexión NVIDIA

Procedimiento: Ejecutar test_nvidia.py en entorno controlado, con clave válida.

Criterio de aceptación: Respuesta HTTP y contenido registrado sin exponer secretos.

Estado: Pendiente de ejecución documentada.

PT-02 · Groq y rotación

Procedimiento: Simular agotamiento o rechazo de una clave y comprobar el cambio a otra.

Criterio de aceptación: La consulta continúa o informa el fallo sin bloqueo indefinido.

Estado: Pendiente de ejecución documentada.

PT-03 · Respaldo NVIDIA

Procedimiento: Simular indisponibilidad de Groq y verificar la ruta alternativa.

Criterio de aceptación: Se registra proveedor utilizado y resultado.

Estado: Pendiente de ejecución documentada.

PT-04 · Agentes secuenciales

Procedimiento: Procesar documento de prueba y registrar 14 inicios y resultados.

Criterio de aceptación: Secuencia completa, errores visibles y resultados asociados.

Estado: Pendiente de ejecución documentada.

PT-05 · Persistencia vectorial

Procedimiento: Consultar varias veces tras la indexación inicial.

Criterio de aceptación: Se recupera información relevante sin reindexar el archivo.

Estado: Pendiente de ejecución documentada.

PT-06 · Rendimiento

Procedimiento: Repetir cargas con documentos de distinto tamaño y medir cada fase.

Criterio de aceptación: Registro de tiempos por fase, total y condiciones de prueba.

Estado: Pendiente de ejecución documentada.

PT-07 · Permisos

Procedimiento: Intentar acceder a documentos con perfiles no autorizados.

Criterio de aceptación: Acceso denegado sin filtración de información.

Estado: Pendiente de ejecución documentada.

PT-08 · Recuperación ante fallos

Procedimiento: Provocar timeout y reiniciar o reintentar según corresponda.

Criterio de aceptación: Error comprensible, sin duplicación de tareas ni espera indefinida.

Estado: Pendiente de ejecución documentada.

6. Registro de incidencias y limitaciones

INC-01 · Consultas detenidas por indisponibilidad de API

El equipo reporta que algunas consultas pueden permanecer en espera cuando un proveedor no responde y que es necesario repetir la solicitud manualmente. Aunque ia_config.py contiene mecanismos de reintento, la experiencia descrita demuestra que la recuperación de extremo a extremo necesita nuevas pruebas y posibles ajustes.

INC-02 · Diferencia entre tiempo estimado y real

La interfaz muestra una duración estimada cercana a 13 minutos; en las pruebas informadas, el flujo completo suele tardar alrededor de 6 minutos. Se requiere medir la diferencia y calibrar la estimación presentada al usuario.

Límites de evidencia

No se adjuntaron capturas de consola, registros de ejecución, reportes de cobertura, actas de aprobación ni resultados de pruebas con usuarios. Por lo tanto, los estados de este documento no deben interpretarse como certificación del sistema ni como evidencia de ejecución automatizada integral.

7. Procedimiento de registro de resultados

Para cada ejecución se recomienda conservar: identificador del caso, fecha y responsable, versión del código, entorno, documento de prueba y tamaño, pasos, resultado esperado, resultado obtenido, duración, evidencia asociada y estado final (aprobado, fallido o bloqueado). Las capturas de consola deben ocultar claves, tokens y datos confidenciales.

8. Conclusiones

La revisión confirma la existencia de una prueba técnica de conexión con NVIDIA y de lógica de resiliencia y validación en el código fuente. No se encontró una suite formal de pruebas unitarias o de integración automatizadas en el material entregado. La verificación sintáctica de los nueve módulos Python fue satisfactoria, pero no sustituye una prueba funcional.

Las pruebas manuales informadas permiten describir el flujo de carga, indexación y análisis de 14 agentes, junto con tiempos aproximados y una incidencia conocida de recuperación ante fallos de API. Para una evaluación formal se requiere incorporar evidencias de ejecución, repetir los casos críticos y documentar resultados reproducibles.

Se propone adoptar los ocho casos complementarios como plan de verificación para los próximos ciclos Scrum, priorizando fallos de proveedor, rotación y respaldo, persistencia vectorial, permisos y mediciones de rendimiento.

9. Control documental

Proyecto: DIRECTA V2 – Módulo Gerente General.
Fecha: 25 de octubre de 2026.
Versión: 1.0.
Integrantes: Rodrigo Bello, Ignacio González, Benjamín Gutiérrez y Javiera Villalobos.
Estado: Informe de revisión estática y plan de pruebas; validación de ejecución pendiente.